講義 7：選擇畫面（Selection Screen）—— PARAMETERS / SELECT-OPTIONS / IN

> 對應練習：ex07 | 答案程式：ZR_TR07_SELSCREEN

本講重點

- 選擇畫面（selection screen）是什麼、何時產生
- PARAMETERS：單值輸入與各種附加選項
- SELECT-OPTIONS：範圍條件，理解背後的 range 表（SIGN/OPTION/LOW/HIGH）
- IN 運算子：range 條件用在 SELECT、LOOP、IF
- SELECTION-SCREEN BLOCK 畫面排版與標題

1. 選擇畫面是什麼

在宣告區寫下 PARAMETERS 或 SELECT-OPTIONS，系統就**自動**產生一個輸入畫面（standard selection screen），執行程式時先顯示，使用者填完按執行（F8）才進主邏輯。不用自己畫畫面——這是傳統報表開發效率高的原因之一。

命名限制：畫面欄位名**最長 8 個字元**。慣例：p_ 開頭是 PARAMETERS、s_ 開頭是 SELECT-OPTIONS。

2. PARAMETERS：單值輸入

```
PARAMETERS p_title TYPE c LENGTH 20 DEFAULT '學生成績清單'.
PARAMETERS p_carr  TYPE scarr-carrid OBLIGATORY.     " 必填
PARAMETERS p_desc  AS CHECKBOX.                      " 核取方塊
PARAMETERS p_file  TYPE string LOWER CASE.           " 保留小寫
```

附加選項	效果
DEFAULT 值	預設值
OBLIGATORY	必填，空白不能執行（欄位出現勾勾）
AS CHECKBOX	核取方塊；勾選時值為 'X'，未勾為空白
LOWER CASE	不自動轉大寫（檔名、密碼類必加，否則輸入被轉成大寫）
RADIOBUTTON GROUP g	單選鈕（同 GROUP 互斥，擇一為 'X'）

```
* 單選鈕：輸出方式擇一
PARAMETERS: p_list AS CHECKBOX,                      " 對照：checkbox 可複選
            p_alv  RADIOBUTTON GROUP g1 DEFAULT 'X',
            p_txt  RADIOBUTTON GROUP g1.

* checkbox / radiobutton 選取時值都是 'X'，沒選是空白
IF p_list = 'X'.        " checkbox：只看自己有沒有勾
```

```
  WRITE / '要印清單'.
ENDIF.

IF p_alv = 'X'.        " radiobutton：同一組一定剛好有一個是 'X'
  WRITE / '用 ALV 輸出'.
ELSEIF p_txt = 'X'.
  WRITE / '輸出成文字檔'.
ENDIF.
```

3. SELECT-OPTIONS：範圍條件

單值不夠用：使用者常要「85 到 100」「A 開頭」「這三個代碼」「排除某段」。SELECT-OPTIONS 一行搞定，畫面自動出現「低值 ~ 高值」兩欄與多重選擇按鈕：

```
DATA gv_score TYPE i.                  " 需要一個「參考欄位」決定型別
SELECT-OPTIONS s_score FOR gv_score.
```

`FOR` 後面必須是**已宣告的資料物件**（變數或 `TABLES` 表工作區的欄位），不能直接寫型別。

3.0 舊程式常見的 `TABLES`：跟表同名的工作區（Table Work Area）

```
TABLES sflight.                          " 宣告一個叫 sflight 的結構變數
SELECT-OPTIONS s_carr FOR sflight-carrid.
```

`TABLES dbtab.` 會宣告一個**跟 DDIC 表（或結構、View）同名、同結構的資料物件**，稱為「表工作區」（table work area），效果大約等於 `DATA sflight TYPE sflight.`。它就是一個跟表同名的結構變數，不是表本身，也不會去讀資料庫。

`TABLES` 後面放的是 DDIC 裡「有欄位結構」的物件：透明表（Transparent Table）、Structure、View 都可以。這個關鍵字叫 `TABLES` 是歷史名稱，不代表只能放資料表。例如：

寫法	後面放的是
`TABLES scarr.`	透明表（資料庫裡真的有這張表）
`TABLES sscrfields.`	Structure（只有欄位定義，資料庫裡沒有這張表）
`TABLES sflights.`	Database View（講義 25 會介紹）

那選擇畫面的輸入，是靠 `TABLES` 傳進程式的嗎？**不是**。`PARAMETERS p_carr` 和 `SELECT-OPTIONS s_carr` 會自己建出 `p_carr`、`s_carr` 這兩個變數，並跟畫面欄位連結，使用者輸入的值就放在這兩個變數裡。`FOR` 後面的 `sflight-carrid` 或 `gv_carrid` 只是提供型別（長度、F1、F4），輸入的值不會放進去。所以一般的選擇畫面，用 `TABLES` 或用 `DATA` 當 `FOR` 的參考，效果都一樣。

`TABLES` 真正不可少的情況，是畫面上有**直接取自 DDIC 結構、名稱是「結構-欄位」**的欄位（官方文件：`TABLES` 關鍵字說明）。這種欄位要靠程式裡同名的 `TABLES` 工作區傳遞資料：畫面顯示前，工作區的值帶到畫面；使用者操作後，畫面的值帶回工作區。選擇畫面上最常碰到的就是下面的 `sscrfields`；之後自己畫 Dynpro 畫面時也會遇到。

舊程式常用它的原因，是 SELECT-OPTIONS ... FOR 後面要一個「已存在的資料物件」，寫一行 TABLES 之後，表裡每個欄位都能直接拿來當參考（FOR sflight-carrid、FOR sflight-fldate……），選擇畫面也會帶出該欄位的說明與 F4。不用 TABLES 的等效寫法：

```
DATA gv_carrid TYPE sflight-carrid.      " 參考 DDIC 欄位的型別
SELECT-OPTIONS s_carr FOR gv_carrid.     " F4、欄位說明一樣會有
```

	TABLES sflight.	DATA gv_carrid TYPE sflight-carrid.
宣告出來的東西	整個結構，名稱跟表一樣	單一欄位變數，名稱自訂
名稱會不會跟表搞混	會：程式裡的 sflight 有時指表、有時指這個變數	不會
官方定位（TABLES 官方文件）	只建議用在跟傳統畫面（dynpro）交換資料，例如講義 28 的 TABLES sscrfields.；Class 裡不能用	一般宣告的標準寫法

維護舊程式時還會看到一個相關寫法：SELECT SINGLE * FROM usr21 WHERE bname = ...，**沒有寫 INTO**。這是因為程式前面有 TABLES usr21.，讀到的資料會自動放進同名的表工作區，之後直接用 usr21-persnumber 取值（講義 13 的 ZRFI0004 就是這樣寫）。這也是舊式寫法，自己寫時一律明確寫出 INTO。

一定要用 TABLES 的情況：TABLES sscrfields.

選擇畫面工具列按鈕的文字、使用者按下的 Function Code，是放在以 DDIC 結構 SSCRFIELDS 定義的畫面欄位裡（例如 SSCRFIELDS-UCOMM）。這正是上面說的「取自 DDIC 結構」的欄位，所以程式要用 TABLES sscrfields. 宣告同名工作區，兩邊才會對上：

- 在 INITIALIZATION 填 sscrfields-functxt_01，按鈕文字才會顯示在畫面上。
- 在 AT SELECTION-SCREEN 讀 sscrfields-ucomm，才知道使用者按了哪顆按鈕。

這裡不能改寫成 DATA sscrfields TYPE sscrfields.：按鈕、功能碼不是 PARAMETERS / SELECT-OPTIONS 建的欄位，程式裡沒有別的變數會跟它們連結，只有 TABLES 工作區才會。選擇畫面加按鈕的完整寫法見講義 13（ZRFI0004 實戰閱讀）與講義 28 第 7 節。

課程的慣例：選擇畫面可以用 TABLES 當 FOR 的參考（講義 13 的範例就是），但除此之外不要拿表工作區來存資料，資料一律放在自己宣告的 gs_ / gt_ 變數。

3.1 背後是一張 range 內表（Selection Table）

s_score 其實是一張內表，每列四個欄位——理解這個結構，就理解了 SELECT-OPTIONS 的一切：

欄位	意義	常見值
SIGN	包含或排除	I（include）/ E（exclude）
OPTION	比較方式	EQ 等於、BT 區間、CP 樣式（含 *）、GE/LE/GT/LT、NE
LOW	低值（或單值）	
HIGH	高值（BT 才用）	

使用者在畫面輸入「85 ~ 100」，系統就往 `s_score` 塞一列 `I / BT / 85 / 100`；多重選擇裡的每一行輸入都是一列。程式也可以自己塞（常用於 INITIALIZATION 給預設範圍）：

```
DATA gs_score LIKE LINE OF s_score.     " 跟 range 表一列同型別

INITIALIZATION.
  gs_score-sign   = 'I'.
  gs_score-option = 'BT'.
  gs_score-low    = 0.
  gs_score-high   = 100.
  APPEND gs_score TO s_score.
```

`LIKE LINE OF` 的意思是「跟這張內表**一列**長一樣」：

寫法	宣告出來的是
`DATA gt_x LIKE s_score.`	整張表（跟 `s_score` 一樣是內表）
`DATA gs_x LIKE LINE OF s_score.`	一列（結構，有 `sign`、`option`、`low`、`high` 四個欄位）

這裡要用 `LIKE LINE OF` 的原因：`s_score` 是 `SELECT-OPTIONS` 自動產生的，程式裡沒有一個具名的型別可以寫 `TYPE`，只能「照著它長」。拿到一列同型別的結構後，就能用講義 4 學的 `APPEND` 塞進去。

這跟講義 2 的 `LIKE` 是同一個觀念：參考**既有的資料物件**；只是多了 `LINE OF`，取的是內表的「一列」而不是整張表。

3.2 另外兩個常見附加項：NO-DISPLAY 與 MATCHCODE OBJECT

舊程式常看到這兩個附加項（講義 13 的實戰案例 ZRFI0004 都有用到）：

```
SELECT-OPTIONS: s_usnam FOR bkpf-usnam MATCHCODE OBJECT user_addr,  " 欄位掛 Search Help
                s_bstat FOR bkpf-bstat NO-DISPLAY.                  " 不顯示在畫面上
```

附加項	效果	典型用途
`MATCHCODE OBJECT sh`	欄位的 F4 改用指定的 Search Help（`user_addr` 是依姓名找 SAP 帳號的標準 Search Help）	欄位本身沒有合適的 F4，或想換成更好用的查詢畫面（Search Help 本身在講義 23 教）
`NO-DISPLAY`	欄位照樣存在、照樣能用 `IN`，但使用者看不到也不能填	程式內部自己組條件（如 ZRFI0004 把三個勾選框轉成文件狀態的 range 表 `s_bstat`），或讓別的程式用 `SUBMIT ... WITH` 傳值進來

ZRFI0004 的做法值得學：畫面上給使用者三個好懂的勾選框（過帳文件 / 暫存文件 / 被刪除的暫存文件），程式在 START-OF-SELECTION 依勾選結果把對應的狀態碼一列列 `APPEND` 進 `NO-DISPLAY` 的 `s_bstat`，最後 SELECT 只要寫一句 `AND bstat IN s_bstat`，不用自己組一堆 `OR`。

4. IN：套用 range 條件

`IN` 判斷「值是否符合 range 表的所有條件」，三個場景通用：

```
* 1) SELECT 的 WHERE
SELECT * FROM sflight INTO TABLE gt_flights
  WHERE carrid IN s_carr.

* 2) LOOP 的 WHERE
LOOP AT gt_students INTO gs_student WHERE score IN s_score.
  WRITE: / gs_student-id, gs_student-name, gs_student-score.
ENDLOOP.

* 3) IF
IF gs_student-score IN s_score.
  ...
ENDIF.
```

關鍵行為：range 表是**空的**（使用者什麼都沒填）時，`IN` 對所有值都成立 = 不過濾。所以「不填就是查全部」不用另外寫 IF 判斷，這是 SELECT-OPTIONS 的預設哲學。

5. 畫面排版：BLOCK 與標題

```
SELECTION-SCREEN BEGIN OF BLOCK b1 WITH FRAME TITLE t_b1.
  PARAMETERS p_title TYPE c LENGTH 20 DEFAULT '學生成績清單'.
  SELECT-OPTIONS s_score FOR gv_score.
  PARAMETERS p_desc AS CHECKBOX.
SELECTION-SCREEN END OF BLOCK b1.

INITIALIZATION.
  t_b1 = '查詢條件'.        " 框標題在 INITIALIZATION 給值
```

- `BLOCK ... WITH FRAME` 把相關欄位框在一起，`TITLE t_xx` 是框標題變數，在 INITIALIZATION 事件裡賦值（事件詳見講義 10）。
- 欄位左邊的說明文字正式做法是 **Selection Texts**（SE38 → Goto → Text Elements → Selection Texts），支援多語言；課堂練習先用預設顯示變數名即可。

5.1 為什麼 `t_b1` 不用 `DATA` 宣告

`TITLE` 後面寫的名稱，系統會**自動產生一個同名的全域變數**，型別是 `c`、長度 70（官方文件：`SELECTION-SCREEN BLOCK` 說明）。這跟 `PARAMETERS p_title` 會自動產生變數 `p_title` 是同一個道理：選擇畫面的宣告本身就會建出變數。所以：

- 程式裡不需要、也不要再寫 `DATA t_b1 ...`。
- 名稱最多 8 個字元（跟 `PARAMETERS` 名稱的限制一樣）。
- 如果 `TITLE` 後面寫的是 `text-001`，就不會產生變數，標題直接取 Text Symbol 的文字（見 5.2）。

為什麼 `PARAMETERS` 要寫 `TYPE`，`TITLE` 卻不用、也不能寫？因為兩者用途不同：

	`PARAMETERS p_title TYPE ...`	`TITLE t_b1`
用途	讓使用者**輸入**資料	只**顯示**框上的標題文字
內容	可能是日期、數字、公司代碼……	永遠是一段文字
型別	由程式用 `TYPE` 指定，決定欄位長度、輸入檢查、F4（沒寫 `TYPE` 時預設 `c` 長度 1）	語法不提供 `TYPE`，固定 `c` 長度 70

輸入欄位的型別會影響畫面怎麼檢查輸入，所以要讓程式指定；框標題只是一行說明文字，沒有其他可能，系統直接給一個固定的文字型別。

5.2 框標題要能多語言：用 Text Symbol

⚠️ 上面 `t_b1 = '查詢條件'.` 把中文寫死在程式裡，不管用哪種語言登入都顯示「查詢條件」。正式程式要用 **Text Symbol**（文字符號，講義 22 詳解），它可以依登入語言（Logon Language）各自維護一份文字：

```
" 寫法一：標題直接用 Text Symbol，不需要 INITIALIZATION
SELECTION-SCREEN BEGIN OF BLOCK b1 WITH FRAME TITLE text-001.

" 寫法二：保留變數，在 INITIALIZATION 指定 Text Symbol
INITIALIZATION.
  t_b1 = text-001.
```

`text-001` 的內容在 SE38 → Goto → Text Elements → **Text Symbols** 維護（編號 `001`、文字「查詢條件」），再翻譯成其他語言。框標題固定不變時用寫法一最簡單；標題要依條件換成不同文字時才用寫法二。本講的練習先用寫死的字串即可。

6. 常見錯誤與陷阱

症狀	原因
SELECT-OPTIONS 報「欄位未定義」	`FOR` 後面的參考變數沒先 DATA 出來
欄位名報錯	超過 8 字元
輸入的小寫字母全變大寫	沒加 `LOWER CASE`
checkbox 判斷 `= 'x'` 不成立	勾選值是大寫 `'X'`
沒輸入條件卻以為會查不到資料	空 range 表 = 全部成立，是規格不是 bug
想在程式裡直接改 s_score 某列	記得它就是內表，APPEND/DELETE/LOOP 都適用

7. 課堂練習

完成 ex07：建 BLOCK 畫面（標題參數、成績範圍、排序 checkbox），用 `IN` 過濾學生名單並處理「查無資料」訊息。